

CUADRO 2.4
Operadores

Operadores de comparación	
<code>e1 === e2</code>	Idéntico. Verdadero si las dos expresiones son del mismo tipo y tienen el mismo valor.
<code>e1 == e2</code>	Igual. Verdadero si las dos expresiones son iguales tras la conversión de tipos, si es necesaria.
<code>e1 !== e2</code>	No idéntico.
<code>e1 != e2, e1 <> e2</code>	No igual.
<code>e1 >= e2, e1 > e2, e1 <= e2, e1 < e2</code>	Mayor o igual, mayor, menor o igual, menor.
<code>e1 ?? e2 ?? e3</code>	Comenzando por la izquierda, devuelve la primera expresión no nula.
Operadores aritméticos	
<code>+e1</code>	Como operador unario, sirve para convertir la expresión a <i>integer</i> o <i>float</i> , según corresponda.
<code>-e1</code>	Como operador unario, cambio de signo.
<code>e1 + e2, e1 - e2, e1 * e2, e1 / e2</code>	Suma, resta, multiplicación, división.
<code>e1 % e2</code>	Módulo.
<code>e1 ** e2</code>	Potencia.
Operadores lógicos	
<code>e1 and e2, e1 && e2</code>	Y. Verdadero si las dos expresiones se evalúan a TRUE.
<code>e1 or e2, e1 e2</code>	O. Verdadero si una o las dos expresiones se evalúan a TRUE.
<code>e1 xor e2</code>	O exclusivo. Verdadero si solo una de las expresiones se evalúa a TRUE.
<code>!e1</code>	Devuelve TRUE si la expresión es FALSE y viceversa.
Operadores a nivel de bit	
<code>e1 & e2, e1 e2, e1 ^ e2, ~e1</code>	Y, O, O exclusivo y negación.
<code>\$var » N</code>	Desplaza N bits de \$var hacia la izquierda.
<code>\$var « N</code>	Desplaza N bits de \$var hacia la derecha.
Operadores de asignación	
<code>\$var1 = e1</code>	Asignación por valor.
<code>\$var2 = &\$var2</code>	Asignación por referencia.
<code>\$var += e1, \$var -= e1, \$var *= e1, \$var /= e1</code>	Equivalentes a <code>\$var = \$var + e1</code> , <code>\$var = \$var - e1</code> ... Válido para cualquier operador binario aritmético, de cadenas o arrays.
Otros operadores	
<code>\$var++, \$var--</code>	Devuelve \$var, luego le suma (resta) 1.
<code>++\$var, --\$var</code>	Suma (resta) 1 a \$var, devuelve el valor actualizado.
<code>\$cad1. \$cad2</code>	Concatena dos cadenas.

2.7. Arrays

Los *arrays* en PHP son una estructura muy flexible y potente. Unifica en un solo tipo lo que en otros lenguajes se consigue con *arrays* básicos, vectores, listas o diccionarios. Los elementos de un *array* se identifican por una clave, que puede ser un entero o una cadena. Los elementos guardan un orden dentro del *array*. Este orden está determinado por el orden de los elementos al declarar el *array* o al añadir nuevos.

Para declarar un *array* se puede utilizar la función `array()`.

```
$var = array(
    clave1 => valor1,
    ...
    claven => valorn
);
```

También es posible utilizar la notación de corchetes

```
$arr = [
    clave1 => valor1,
    ...
    claven => valorn
];
```

Se puede acceder a cada elemento del *array* por su clave utilizando los corchetes, como es habitual en otros lenguajes:

```
$arr[clave] = valor;
```

El ejemplo **arrays1.php** muestra cómo declarar y utilizar los *arrays*. En el ejemplo se utiliza la función `print_r()`, que muestra información sobre la variable que se le pase como argumento. Si se le pasa un *array*, muestra las claves y valores de todos los elementos.

```
<?php
    $arr1 = [
        0 => 444,
        1 => 222,
        2 => 333,
    ];
    print_r($arr1);
    echo "<br>". "pos 0: ". $arr1[0]. "<br>";
    $arr1[0] = 555;
    print_r($arr1);
    echo "<br>";
    $arr2 = array (
        "1111A" => "Juan Vera Ochoa",
        "1112A" => "Maria Mesa Cabeza",
        "1113A" => "Ana Puertas Peral"
    );
    $arr2["1113A"] = "Ana Puertas Segundo";
```

Para recorrer un *array* lo habitual es utilizar el bucle `foreach`.


```
foreach($arr as $valor){
    ...
}
```

El cuerpo del bucle se ejecuta una vez por cada elemento del *array* `$arr`. En cada iteración del bucle `$valor` almacena el elemento correspondiente.

También se puede utilizar especificando una variable para la clave,

```
foreach($arr as $clave => $valor){ ... }
```

En este caso, en cada iteración también se dispondrá de la clave del elemento correspondiente en la variable `$clave`.

```
<?php
    $arr2 = array (
        "1111A" => "Juan Vera Ochoa",
        "1112A" => "Maria Mesa Cabeza",
        "1113A" => "Ana Puertas Peral"
    );
    foreach ($arr2 as $nombre) {
        echo "$nombre <br>";
    }
    foreach ($arr2 as $codigo => $nombre) {
        echo "Código: $codigo Nombre: $nombre <br>";
    }
}
```

Si se pretende modificar el contenido del *array* al recorrerlo con un bucle `foreach`, hay que utilizar una referencia al declarar las variables del bucle. En el ejemplo **foreach_modificar.php**, el primer bucle no utiliza referencias y, por tanto, no modifica el *array*. El segundo bucle es la forma correcta de hacerlo.

```
<?php
    $arr1 = array(
        "Viernes" => 22,
        "Sábado" => 34
    );
    /* no modifica el array */
    foreach ($arr1 as $cantidad) {
        $cantidad = $cantidad * 2;
    }
    print_r($arr1);
    echo "<br>";
    /* modifica el array */
    foreach ($arr1 as &$cantidad) {
        $cantidad = $cantidad * 2;
    }
    print_r($arr1);
```

La salida del programa será:

```
Array ( [Viernes] => 22 [Sábado] => 34 )
Array ( [Viernes] => 44 [Sábado] => 68 )
```

También es posible declarar un *array* sin usar claves. En ese caso, se asignan como claves por defecto enteros consecutivos empezando por 0. Si se añade un nuevo elemento sin especificar clave, la clave será el número siguiente a la mayor clave entera presente en el *array*.

```
<?php
$arr1 = array(10, 20, 30, 40);
print_r($arr1);
echo "<br>";
$arr1[] = 5;
print_r($arr1);
echo "<br>";
$arr1[10] = 6;
$arr1[] = 5;
print_r($arr1);
echo "<br>";
```

Salida:

```
Array ( [0] => 10 [1] => 20 [2] => 30 [3] => 40 )
Array ( [0] => 10 [1] => 20 [2] => 30 [3] => 40 [4] => 5 )
Array ( [0] => 10 [1] => 20 [2] => 30 [3] => 40 [4] => 5 [12] => 6 [11] => 5 [13] => 5 )
```

Algunos operadores también están disponibles para los *arrays*, pero su significado no es exactamente el mismo. Por ejemplo, el operador "Idéntico" solo será verdadero si los dos *arrays* tienen todos los elementos iguales tanto en clave como en valor y además están en el mismo orden. Para el operador "Igual", el orden de las claves no importa.

```
<?php
$arr1 = array(
    1 => "3000",
    2 => "4000",
);
$arr2 = array(
    1 => 3000,
    2 => 4000,
);
$arr3 = array(
    2 => "4000",
    1 => "3000",
);
if($arr1 == $arr2){
    echo "arr1 y arr2 son iguales <br>";
}else{
    echo "arr1 y arr2 no son iguales <br>";
}
if($arr1 == $arr3){
    echo "arr1 y arr3 son iguales <br>";
}else{
    echo "arr1 y arr3 no son iguales <br>";
}
if($arr1 === $arr2){
    echo "arr1 y arr2 son idénticos <br>";
}else{
    echo "arr1 y arr2 no son idénticos <br>";
}
if($arr1 === $arr3){
    echo "arr1 y arr3 son idénticos <br>";
}else{
    echo "arr1 y arr3 no son idénticos <br>";
}
```


La salida será:

arr1 y arr2 son iguales
 arr1 y arr3 son iguales
 arr1 y arr2 no son idénticos
 arr1 y arr3 son idénticos

El operador "+" utilizado con dos *arrays* devuelve su unión. El *array* resultante contendrá primero los elementos del primer *array* (el que aparece a la izquierda del operador) y a continuación los del segundo, pero sin repetir claves. Si hay elementos con la misma clave en los dos *arrays*, el del segundo no se añade al resultado.

```
<?php
    $arr1 = array(
        10 => "3000",
        20 => "4000",
        30 => "6000",
    );
    print_r($arr1);
    echo "<br>";
    $arr2 = array(
        10 => "8000",
        15 => "6000",
        20 => "4000",
    );
    print_r($arr2);
    echo "<br>";
    $arr3 = $arr1 + $arr2;
    print_r($arr3);
```

El resultado será:

```
Array ( [10] => 3000 [20] => 4000 [30] => 6000 )
Array ( [10] => 8000 [15] => 6000 [20] => 4000 )
Array ( [10] => 3000 [20] => 4000 [30] => 6000 [15] => 6000 )
```

CUADRO 2.5
Operadores para *arrays*

Operador	Descripción
\$a1 === \$a2	Idéntico. Verdadero si los dos <i>arrays</i> tienen las claves y valores iguales, en el mismo orden y del mismo tipo.
\$a1 !== \$a2	No idéntico.
\$a1 == \$a2	Igual. Verdadero si los dos <i>arrays</i> tienen las claves y valores iguales.
\$a1 != \$a2, \$a1 <> \$a2	No igual.
\$a1 + \$a2	Unión. Devuelve un <i>array</i> con los elementos de ambos.

2.8. Funciones y librerías

Una función es un conjunto de instrucciones que realiza una tarea concreta. Las funciones pueden recibir argumentos, los datos que necesitan para llevar a cabo su cometido. Por ejemplo, una función que abre un fichero puede recibir como argumento el nombre del fichero que tiene que abrir.

Las funciones pueden devolver o no un valor. A diferencia de lo que ocurre en otros lenguajes, en PHP no es necesario declarar el tipo de dato que devuelve una función y, de hecho, puede devolver tipos de datos diferentes según el caso. Por ejemplo, en PHP hay muchas funciones que devuelven FALSE en caso de error y no devuelven un *boolean* si no hay error.

2.8.1. Funciones predefinidas

PHP cuenta con una gran cantidad de funciones predefinidas para las tareas más habituales.

Entre las más utilizadas están las relacionadas con las variables. Como en PHP las variables no se declaran explícitamente, hay una serie de funciones que permiten conocer el estado y el tipo de dato de una variable. Las más importantes son:

- `is_null($var)`. Devuelve TRUE si `$var` es NULL, FALSE en otro caso.
- `isset($var)`. Devuelve TRUE si `$var` ha sido inicializada y su valor no es NULL, FALSE en otro caso.
- `unset($var)`. Elimina la variable. Ya no contará como inicializada.
- `empty($var)`. Devuelve TRUE si `$var` no ha sido inicializada o su valor es FALSE, FALSE en otro caso.
- `is_int($var)`, `is_float($var)`, `is_bool($var)`, `is_array($var)`. Devuelven TRUE si `$var` es entero, *float*, *booleano* o *array* respectivamente, y FALSE en otro caso.
- `print_r($var)` y `var_dump($var)`. Muestran información sobre `$var`.

En el ejemplo **funciones_variables.php** se pueden ver las diferencias entre estas funciones.

```
<?php
    $var1 = 4;
    $var2 = NULL;
    $var3 = FALSE;
    $var4 = 0;
    echo "var 1";
    var_dump(isset($var1)); // TRUE
    var_dump(is_null($var1)); // FALSE
    var_dump(empty($var1)); // FALSE
    echo "var 2";
    var_dump(isset($var2)); // FALSE
    var_dump(is_null($var2)); // TRUE
    var_dump(empty($var2)); // TRUE
    echo "var 3";
    var_dump(isset($var3)); // TRUE
    var_dump(is_null($var3)); // FALSE
    var_dump(empty($var3)); // TRUE
    echo "var 4";
```



```

var_dump(empty($var4)); // TRUE, EL 0 COMO BOOLEAN ES FALSE
echo "unset";
unset($var1);
var_dump(isset($var1)); // FALSE

```

A lo largo de los siguientes capítulos se irán presentando más funciones según sean necesarias. En el cuadro 2.6 se resumen algunas de las más útiles.

CUADRO 2.6
Algunas funciones útiles en PHP

Funciones de variables	
isset(\$var)	TRUE si la variable está inicializada y no es NULL
is_null(\$var)	TRUE si la variable es NULL
empty(\$var)	TRUE si la variable no está inicializada o su valor es FALSE
is_int(\$var), is_float(\$var), is_bool(\$var), is_array(\$var)	Para comprobar el tipo de dato de \$var
intval(\$var), floatval(\$var), boolvar(\$var), strval(\$var)	Para obtener el valor de \$var como otro tipo de dato
Funciones de cadenas	
strlen(\$cad)	Devuelve la longitud de \$cad
explode(\$cad, \$token)	Parte una cadena utilizando \$token como separador. Devuelve un array de cadenas
implode(\$token, \$array)	Crea una cadena larga a partir de un array de cadenas, entre cadena y cadena se introduce \$token
strcmp(\$cad1, \$cad2)	Compara las dos cadenas. Devuelve 0 si son iguales, -1 si \$cad1 es menor y 1 si \$cad1 es mayor
strtolower(\$cad), strtoupper(\$cad)	Devuelven \$cad en mayúsculas o minúsculas, respectivamente
str(\$cad1, \$cad2)	Busca la primera ocurrencia de \$cad2 en \$cad1. Si no aparece devuelve FALSE, si aparece devuelve \$cad1 desde donde comienza la ocurrencia
Funciones de arrays	
ksort(\$arr), krsort(\$arr)	Ordena el array por clave en orden ascendente o descendente
sort(\$arr), rsort(\$arr)	Ordena el array por valor en orden ascendente o descendente
array_values(\$arr)	Devuelve los valores de \$arr
array_keys(\$arr)	Devuelve las claves de \$arr
array_key_exists(\$arr, \$cla)	Devuelve verdadero si algún elemento de \$arr tiene clave \$cla
count(\$arr)	Devuelve el número de elementos del array

2.8.2. Funciones definidas por el usuario

La sintaxis para definir una función es:

```
function nombre(argumentos){  
    <instrucciones>  
}
```

La función puede tener o no argumentos. Se declaran en la cabecera de la función, entre los paréntesis, separados por comas. Los argumentos son variables locales a la función y solo existen mientras se esté ejecutando. En PHP no hace falta declarar el tipo de dato que devuelve la función. De hecho, es posible que una función devuelva tipos de datos diferentes según el caso.

Por ejemplo, la siguiente función recibe dos números y devuelve su suma.

```
<?php  
function suma($a, $b) {  
    return $a + $b;  
}  
echo suma(4,8). '<br>';  
$var1 = 35;  
$var2 = 5;  
$var3 = suma($var1, $var2);  
echo $var3. '<br>';
```

Es posible especificar valores por defecto para los argumentos. Si al llamar a la función no se usa el argumento, se toma el valor por defecto.

```
<?php  
function saludar($nombre = 'usuario'){  
    echo "Hola $nombre <br>";  
}  
saludar();  
saludar("Ana");
```

La salida será:

Hola usuario
Hola Ana

Actividad propuesta 2.2



Escribe una función para calcular potencias. Recibirá como argumentos la base y el exponente, que es opcional y tiene valor por defecto 2 (elevar al cuadrado).

2.8.3. Paso de argumentos por copia y por valor

En PHP los argumentos se pasan por copia. Esto quiere decir que, cuando se llama a una función con una variable como argumento, se crea una variable local a la función en la que se copia el valor del argumento. Por tanto, si la función modifica el argumento, estos cambios no tienen efecto en la variable original.

Esto es lo que ocurre en la función `duplicarMal()` del siguiente ejemplo. El argumento se multiplica por dos, pero el valor de `$var1` no cambia. Para solucionarlo hay dos opciones. Devolver el nuevo valor y reasignar, como se hace con la función `duplicar()`, o utilizar una referencia, como se hace en `duplicar2()`. Para utilizar una referencia solo hace falta añadir el símbolo "&" antes del argumento.

```
<?php
function duplicarMal($a){
    $a = $a *2;
}
function duplicar($a){
    return $a *2;
}
function duplicar2(&$a){
    $a = $a *2;
}
$var1 = 5;
duplicarMal($var1);
echo "$var1 <br>";
$var1 = duplicar($var1);
echo "$var1 <br>";
duplicar2($var1);
echo "$var1 <br>";
```

La salida del programa será:

```
5
10
20
```

2.8.4. Funciones como argumentos

En PHP es posible pasar funciones como argumentos a otras funciones. Es una característica avanzada que se utiliza entre otras cosas para las funciones de *callback*. Solo hay que pasar el nombre de la función entre comillas como argumento. La función que lo recibe podrá utilizarla si le pasan los argumentos adecuados.

En el ejemplo **calculador.php** se utiliza esta característica. La función `calculador()` recibe como argumentos dos números y también la función que debe aplicarles. Según qué función se le pase como argumento, devolverá un valor u otro.

```
<?php
function calculador($operacion, $numa, $numb){
    $resul = $operacion($numa, $numb);
```

```

        return $resul;
    }
    function sumar($a, $b){
        return $a + $b;
    }
    function multiplicar($a, $b){
        return $a * $b;
    }
    $a = 4;
    $b = 5;
    $r1 = calculador("multiplicar", $a, $b);
    echo "$r1 <br>";
    $r2 = calculador("sumar", $a, $b);
    echo "$r2 <br>";

```

Actividad propuesta 2.3



Escribe una función para calcular el factorial de un número, que recibirá como argumento. Devolverá el factorial o -1 si el argumento no es válido.

2.9. Excepciones y errores

El sistema de control de errores de PHP ha ido evolucionando a lo largo de las versiones. En el sistema básico, se generan errores de diferentes tipos, representados por un número. Por otro lado, desde PHP 5 hay un sistema de excepciones similar al de Java y otros lenguajes utilizando bloques `try/catch/finally`. Finalmente, en PHP 7 aparecieron las excepciones de clase `Error`.

2.9.1. Errores

En el sistema básico, ante determinadas condiciones (por ejemplo, utilizar una variable no inicializada) PHP genera un error. Hay diferentes tipos de errores, cada uno asociado con un número y una constante predefinida. Se puede controlar cómo se comporta PHP antes los errores mediante tres directivas del fichero `php.ini`:

- *error_reporting*: indica qué errores deben reportarse. Lo normal es utilizar `E_ALL`, es decir, todos.
- *display_errors*: señala si los mensajes de error deben aparecer en la salida del *script*. Esta opción es apropiada durante el desarrollo, pero no en producción.
- *log_errors*: indica si los mensajes de error deben almacenarse en un fichero. Es especialmente útil en producción, cuando no se muestran los errores en la salida.
- *error_log*: si la directiva anterior está activada, es la ruta en la que se guardan los mensajes de error.

El valor de la directiva *error_reporting* es un número, pero para especificarlo lo habitual es utilizar las constantes predefinidas y el operador *or* a nivel de bit.

CUADRO 2.7
Tipos de error

Código	Constante	Descripción
1	E_ERROR	Error fatal en tiempo de ejecución. La ejecución del <i>script</i> se detiene
2	E_WARNING	Advertencia en tiempo de ejecución. El <i>script</i> no se detiene
4	E_PARSE	Error de sintaxis al compilar
8	E_NOTICE	Notificación. Puede indicar error o no
16	E_CORE_ERROR	Error fatal al iniciar PHP
32	E_CORE_WARNING	Advertencia al iniciar PHP
64	E_COMPILE_ERROR	Error fatal al compilar
128	E_COMPILE_WARNING	Advertencia fatal al compilar
256	E_USER_ERROR	Error generado por el usuario
512	E_USER_WARNING	Advertencia generada por el usuario
1024	E_USER_NOTICE	Notificación generada por el usuario
2048	E_STRICT	Sugerencias para mejorar la portabilidad
4096	E_RECOVERABLE_ERROR	Error fatal capturable
8192	E_DEPRECATED	Advertencia de código obsoleto
16384	E_USER_DEPRECATED	Como la anterior, generada por el usuario
32767	E_ALL	Todos los errores



Actividad propuesta 2.4

Modifica el fichero *php.ini* para que los errores *E_NOTICE* no se muestren por pantalla (*display_errors*).

Comprueba la diferencia accediendo a *localhost/cap2/no_init.php*.

• Funciones relacionadas

La función *error_reporting()* permite cambiar el valor de la directiva *error_reporting* en tiempo de ejecución.

También es posible definir una función propia para que se encargue de los errores utilizando `set_error_handler()`. La función que se ocupe de los errores tendrá que tener la siguiente firma:

```
bool handler ( int $errno, string $errstr [, string $errfile [, int $errline [, array $errcontext ]]] );
```

El siguiente ejemplo muestra cómo utilizar `set_error_handler()` para manejar los errores con una función propia.

```
<?php
function manejadorErrores($errno, $str, $file, $line){
    echo "Ocurrió el error: $errno";
}
set_error_handler("manejadorErrores");
$a = $b; // causa error, $b no está inicializada
```

La salida será:

Ocurrió el error: 8

2.9.2. Excepciones

Otra opción para indicar un error es lanzar una excepción. Para controlar las excepciones se utilizan bloques `try/catch/finally`, como en Java. Cuando se lanza una excepción y no es capturada por un bloque `catch`, la ejecución del programa se detiene. Si es capturada, se ejecuta el código del bloque correspondiente.

Para capturar una excepción se introduce la instrucción que puede causarla dentro de un bloque `try` y se añade el bloque `catch` correspondiente. Se puede añadir un bloque `finally`, que se ejecuta después del `try/catch`, haya habido excepción o no.

```
try{
    instrucciones;
}catch(Exception e){
    instrucciones;
}finally{
    instrucciones;
}
```

En el ejemplo **excepciones.php** se utiliza una función que recibe dos argumentos y lanza una excepción si el segundo es cero. Para lanzar una excepción se utiliza `throw`, que recibe como argumento un objeto de clase `Exception` o de alguna subclase.

```
1  <?php
2      function dividir($a, $b){
3          if ($b==0){
4              throw new Exception('El segundo argumento es 0');
5          }
```



```

6      return $a/$b;
7  }
8  try{
9      $resul1 = dividir(5, 0);
10     echo "Resul 1 $resul1". "<br>";
11 }catch(Exception e){
12     echo "Excepción: ". $e->getMessage(). "<br>";
13 }finally{
14     echo "Primer finally";
15 }
16 try{
17     $resul2 = dividir(5, 2);
18     echo "Resul 2 $resul2". "<br>";
19 }catch(Exception e){
20     echo "Excepción: ". $e->getMessage(). "<br>";
21 }finally{
22     echo "Segundo finally";
23 }

```

En la primera llamada a `dividir()`, línea 9, se produce una excepción. Por tanto, el resto del bloque `try`, el `echo`, no se ejecuta. Sí se ejecutan el `catch` y el `finally` correspondientes.

En la segunda llamada a `dividir()`, línea 17, no se produce excepción. En este caso se ejecutan el `echo` del `try` y a continuación el del bloque `finally`. El bloque `catch` no se ejecuta.



Actividad propuesta 2.5

Adapta la actividad 2.3 para que controle si el argumento es negativo utilizando una excepción.

2.9.3. Excepciones Error

En PHP 7 aparecieron las excepciones de tipo `Error`. No heredan de la clase `Exception`, así que para capturarlas hay que usar:

```

catch (Error $e){
    ...
}

```

o, alternatively, la clase `Throwable`, de la que se derivan tanto `Error` como `Exception`:

```

catch (Throwable $e){
    ...
}

```

En el cuadro 2.8. se muestran las excepciones Error predefinidas.

CUADRO 2.8
Excepciones Error

Nombre	Descripción	Hereda de
Error	Clase base para las excepciones Error	
ArithmeticError	Error en operaciones matemáticas. Hereda del anterior	Error
DivisionByZeroError	Intento de división por cero. Hereda del anterior	ArithmeticError
AssertionError	Ocurre cuando falla una llamada a assert()	Error
ParseError	Error al compilar	Error
TypeError	Ocurre cuando una expresión no tiene el tipo de dato que se espera	Error
ArgumentCountError	Ocurre al llamar a una función con menos argumentos de los necesarios. Hereda del anterior	TypeError

2.10. Clases y objetos

PHP tiene soporte completo para la programación orientada a objetos. Permite definir clases, herencia, interfaces y los demás elementos habituales. Para declarar una clase se utiliza la palabra reservada **class**. Los atributos se declaran utilizando su nombre, los modificadores que pueda tener y opcionalmente un valor por defecto.

```
class Clase{
    private $att1 = 10; // con valor por defecto
    private $atr2; // sin valor por defecto
    private static $atr3 = 0; // estático
    ...
}
```

Los atributos y métodos se pueden declarar como estáticos, de manera que no habrá uno por objeto, sino uno por clase. Se utiliza la palabra reservada **static**.

Se puede crear un constructor para la clase con el método **__construct()**. Este método forma parte de los *métodos mágicos*, nombres reservados a tareas concretas y que comienzan por dos guiones bajos, **__**.

Para crear un nuevo objeto se utiliza el operador **new** seguido del nombre de la clase:

```
$obj = new Clase();
```

Para los atributos y métodos se pueden utilizar los siguientes modificadores de visibilidad:

- **public**. Se pueden utilizar desde dentro y fuera de la clase.
- **private**. Pueden emplearse desde la propia clase.
- **protected**. Se pueden utilizar dentro de la propia clase, las derivadas y las antecesoras.

Normalmente los atributos se declaran como privados y se crean métodos públicos para acceder a ellos. Para acceder a los métodos y atributos se utiliza:

```
$objeto->propiedad;
$objeto->método(argumentos);
```

Los métodos son funciones definidas dentro de una clase. En los métodos no estáticos se puede utilizar la palabra reservada `this`, que representa el objeto desde el que se invoca el método.

El ejemplo **PersonayCliente.php** muestra cómo crear una clase y crear y manejar objetos de esta. Para empezar, declara la clase `Persona`, con atributos para nombre, apellido y DNI. Los tres son privados. La clase tiene un constructor, el método `__construct()`, que recibe valores para los tres atributos. A continuación, se declaran una serie de métodos públicos para leer y escribir los atributos, los llamados *getters* y *setters* (líneas 11-23).

```
1  <?php
2  class Persona {
3      private $DNI;
4      private $nombre;
5      private $apellido;
6      function __construct($DNI, $nombre, $apellido) {
7          $this->DNI = $DNI;
8          $this->nombre = $nombre;
9          $this->apellido = $apellido;
10     }
11     public function getNombre() {
12         return $this->nombre;
13     }
14     public function getApellido() {
15         return $this->apellido;
16     }
17     public function setNombre($nombre) {
18         $this->nombre = $nombre;
19     }
20
21     public function setApellido($apellido) {
22         $this->apellido = $apellido;
23     }
24     public function __toString() {
25         return "Persona: ".$this->nombre." ".$this->apellido;
26     }
27 }
```

En esta clase también se utiliza `__toString()` (líneas 24-26), otro método mágico que se usa para generar una cadena de texto con la información del objeto. Cuando se pasa un objeto a `echo`, se representa usando el valor de retorno de `__toString()`.

En las líneas 46-52 se crea un objeto de la clase `Persona` y se manipula.

```
45 // crear una persona
46 $per = new Persona("1111111A", "Ana", "Puertas");
47 // mostrarla, usa el método __toString()
48 echo $per. "<br>";
49 // cambiar el apellido
50 $per->setApellido("Montes");
```

```
51 // volver a mostrar
52 echo $per. "<br>";
```

La salida de este fragmento será:

```
Persona: Ana Puertas
Persona: Ana Montes
```

Para crear una clase que herede de otra, se utiliza la palabra clave **extends**. La clase derivada tendrá los mismos atributos y métodos que la clase base y podrá añadir nuevos o sobrescribirlos.

En el ejemplo anterior se declara también la clase **Cliente**, que hereda de **Persona** y añade un atributo **saldo**. Incluye también los métodos **getSaldo()** y **setSaldo()** y sobrescribe el método **__toString()** de la clase base. También hay un constructor que incluye el nuevo atributo y utiliza el constructor de la clase base.

```
class Cliente extends Persona{
    private $saldo = 0;
    function __construct($DNI, $nombre, $apellido, $saldo){
        parent::__construct($DNI, $nombre, $apellido);
        $this->$saldo = $saldo;
    }
    public function getSaldo(){
        return $this->saldo;
    }
    public function setSaldo($saldo){
        $this->saldo = $saldo;
    }
    public function __toString(){
        return "Cliente: ". $this->getNombre();
    }
}
```

En las líneas 54-56 se crea un objeto de la clase cliente y se muestra. La salida será "Cliente: Pedro".

```
$cli = new Cliente("22222245A", "Pedro", "Sales", 100);
// lo muestra
echo $cli. "<br>";
```

Resumen

- PHP es el lenguaje más extendido para el desarrollo web en el lado del servidor.
- Los ficheros de PHP mezclan HTML y PHP. El servidor sustituye los bloques de PHP por su salida.
- En PHP no hace falta declarar las variables, se declaran la primera vez que se usan.
- Tampoco es necesario definir su tipo de dato, se infiere del valor de inicialización.

- El paso de parámetros y la asignación se realizan por defecto por copia, pero se pueden usar referencias.
- Las funciones de PHP pueden ser parámetros de otras funciones. Es una característica muy usada en las librerías PHP.
- PHP cuenta con muchas funciones integradas para las tareas más habituales: manipulación de cadenas, *arrays*, bases de datos, ficheros...
- Los *arrays* de PHP son un tipo de dato muy potente similar a los que en otros lenguajes se llaman *mapas* o *diccionarios*.
- Hay tres sistemas de control de errores en PHP: errores, excepciones y excepciones de clase Error.
- PHP tiene soporte completo para la programación orientada a objetos.

Ejercicios propuestos



1. Escribe un *script* para resolver ecuaciones de segundo grado, $ax^2 + bx + c = 0$. Si la ecuación no tiene soluciones reales, hay que mostrar un mensaje de error.
2. Crea una función para resolver la ecuación de segundo grado. Esta función recibe los coeficientes de la ecuación y devuelve un *array* con las soluciones. Si no hay soluciones reales, devuelve FALSE.
3. Almacena la función anterior en el fichero **matemáticas.php**. Crea un fichero que la incluya y la utilice.
4. Escribe una función que reciba una cadena y comprueba si es un palíndromo.
5. Escribe una función que reciba un *array* de números, y un número, el límite. La función tiene que devolver un nuevo *array* que contenga solo los elementos del *array* original menores que el límite.
6. Escribe un *script* para probar las funciones del cuadro 2.6.

ACTIVIDADES DE AUTOEVALUACIÓN

1. En PHP, las variables:
 - ☐ a) No tienen tipo de dato.
 - ☐ b) Tienen un tipo de dato que puede cambiar.
 - ☐ c) Tienen un tipo de dato que no puede cambiar.
2. En PHP, las funciones:
 - ☐ a) Tienen que declarar el tipo de dato que devuelven.
 - ☐ b) Pueden devolver diferentes tipos de datos según el caso.
 - ☐ c) No tienen que declarar el tipo de dato que devuelven, pero siempre tienen que devolver el mismo.

3. En los *arrays* de PHP, si no se asigna clave al insertar un elemento:

- ☐ a) Da error.
- ☐ b) Se inserta sin clave.
- ☐ c) Se le asigna una clave numérica.

4. El orden de los elementos dentro de un *array* está determinado por:

- ☐ a) Su clave.
- ☐ b) Su valor.
- ☐ c) El orden de inserción.

5. Al utilizar una variable no inicializada:

- ☐ a) Se genera un error de tipo E_NOTICE y la ejecución del *script* se detiene.
- ☐ b) Se genera un error de tipo E_NOTICE y la ejecución del *script* continúa.
- ☐ c) No se genera ningún error.

6. La salida de estas líneas de código es:

```
$arr = array(2,2,2,2);  
foreach($arr as $elemento){  
    $elemento = $elemento *2;  
}  
foreach($arr as $elemento){  
    echo $elemento. " ";  
}
```

- ☐ a) 2 2 2 2.
- ☐ b) 4 4 4 4.
- ☐ c) Hay un error.

7. La salida de estas líneas de código es:

```
$a = 4;  
$b = &$a;  
$a = 6;  
echo $b;
```

- ☐ a) 4
- ☐ b) 6
- ☐ c) 8.

8. En PHP, los argumentos:

- ☐ a) Se pasan por copia.
- ☐ b) Se pasan por valor.
- ☐ c) Se pueden pasar por copia o por valor.

9. Sobre los bloques de PHP:

- ☐ a) Se deben cerrar siempre.
- ☐ b) Si el fichero acaba con un bloque, no se cierra.
- ☐ c) Si el fichero contiene solo bloques de PHP, el último no se cierra.

10. Dos *arrays* son idénticos si:

- ☐ a) Todos los elementos tienen el mismo valor y están en el mismo orden.
- ☐ b) Todos los elementos son iguales en clave y valor y además están en el mismo orden.
- ☐ c) Todos los elementos son iguales en clave y valor.

SOLUCIONES:

1. ☐ a ☒ b ☐ c

2. ☐ a ☒ b ☐ c

3. ☐ a ☐ b ☒ c

4. ☐ a ☐ b ☒ c

5. ☐ a ☒ b ☐ c

6. ☒ a ☐ b ☐ c

7. ☐ a ☒ b ☐ c

8. ☐ a ☐ b ☒ c

9. ☐ a ☐ b ☒ c

10. ☐ a ☒ b ☐ c